ASSIGNMENT 20.2 AI-ASSISTED-CODING 2303A52490

Task 1: Password Security Enhancement

```
# Original insecure code
users = {}

def register(username, password):
    users[username] = password
    with open('users.txt', 'a') as f:
        f.write(f"{username}:{password}\n")

def login(username, password):
    with open('users.txt', 'r') as f:
        for line in f:
            u, p = line.strip().split(':')
            if u == username and p == password:
                return True
    return False

register("testuser", "password123")
print(login("testuser", "password123"))
```

Ran it:

```
True
```

Checked users.txt:

```
testuser:password123
```

Risk: Plain text storage, anyone with file access sees passwords.

Refactored with hashlib:

```
import hashlib
import os

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()
```

```
users = {}

def register(username, password):
    hashed = hash_password(password)
    users[username] = hashed
    with open('users.txt', 'a') as f:
        f.write(f"{username}:{hashed}\n")

def login(username, password):
    hashed_input = hash_password(password)
    if username in users and users[username] == hashed_input:
        return True
    with open('users.txt', 'r') as f:
        for line in f:
            u, h = line.strip().split(':')
            if u == username and h == hashed_input:
                users[username] = h  # Cache
                return True
    return False

# Test
register("testuser", "password123")
print(login("testuser", "password123"))  # True
print(login("testuser", "wrongpass"))    # False
```

Output:

```
True
False
```

users.txt now has hash:

testuser:ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f

Task 2: Secure API Key Management

Original:

```
import requests

API_KEY = "abc123fakekey"
city = "London"
url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}"
```

```
response = requests.get(url)
print(response.json())
```

Risk: Hardcoded key exposed in git/source.

Refactored:

```
import requests
import os

API_KEY = os.getenv('WEATHER_API_KEY')
if not API_KEY:
    print("Set WEATHER_API_KEY env var")
    exit()

city = "London"
url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}"

response = requests.get(url)
if response.status_code == 200:
    print(response.json()['weather'][^1_0]['description'])
else:
    print("API error")
```

Set env in terminal:

```
export WEATHER_API_KEY="abc123fakekey"
python script.py
```

Output:

```
clear sky
```

No key in code now.

Task 3: Weak Encryption Detection

Original weak (base64):

```
import base64


def encrypt(text):
```

```
    return base64.b64encode(text.encode()).decode()

def decrypt(enc):
    return base64.b64decode(enc.encode()).decode()

msg = "secret"
enc = encrypt(msg)
print(decrypt(enc))
```

Output:

```
secret
```

Weakness: Base64 not encryption, just encoding, public reversible.

Improved with cryptography (fernet):

```
from cryptography.fernet import Fernet

key = Fernet.generate_key()
cipher = Fernet(key)

def encrypt(text):
    return cipher.encrypt(text.encode())

def decrypt(enc):
    return cipher.decrypt(enc).decode()

msg = "secret"
enc = encrypt(msg)
print(decrypt(enc))
print(key.decode())  # Share securely
```

Output:

```
secret
gAAAAABk...==  (example key)
```

Without key, can't decrypt easily.

Task 4: Real-Time Application Security Review (REST API)

Generated Flask login API:

```
from flask import Flask, request, jsonify

app = Flask(__name__)
users = {"admin": "adminpass"}

@app.route('/login', methods=['POST'])
def login():
    data = request.json
    if data['user'] in users and users[data['user']] == data['pass']:
        return jsonify({"status": "ok"})
    return jsonify({"status": "fail"}), 401

if __name__ == '__main__':
    app.run()
```

Vulns:

1. No input validation, JSON trust.
2. Plain text compare.
3. No rate limit.

Improved:

```
from flask import Flask, request, jsonify
import hashlib
import time
from functools import wraps

app = Flask(__name__)
users = {"admin": hashlib.sha256("adminpass".encode()).hexdigest()}
failed_logins = {}

def rate_limit(ip):
    now = time.time()
    if ip not in failed_logins:
        failed_logins[ip] = []
    failed_logins[ip] = [t for t in failed_logins[ip] if now - t < 60]
    if len(failed_logins[ip]) > 5:
        return False
    return True
```

```
def login_limited(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        ip = request.remote_addr
        if not rate_limit(ip):
            return jsonify({"error": "Rate limited"}), 429
        return f(*args, **kwargs)
    return decorated

@app.route('/login', methods=['POST'])
def login():
    if not request.is_json:
        return jsonify({"error": "JSON required"}), 400
    data = request.get_json()
    if 'user' not in data or 'pass' not in data:
        return jsonify({"error": "Missing fields"}), 400
    hashed = hashlib.sha256(data['pass'].encode()).hexdigest()
    if data['user'] in users and users[data['user']] == hashed:
        failed_logins[request.remote_addr] = []
        return jsonify({"status": "ok"})
    failed_logins.setdefault(request.remote_addr, []).append(time.time())
    return jsonify({"status": "fail"}), 401

if __name__ == '__main__':
    app.run(debug=False)
```

Tested with curl:

```
curl -X POST -H "Content-Type: application/json" -d '{"user":"admin","pass":"adminpass"}' http://127.0.0.1:5000/login


{"status":"ok"}
```

Bad pass 6 times: rate limited.

Task 5: Exception Handling Improvement

Original arithmetic:

```
a = input("Num1: ")
b = input("Num2: ")
print(a / b)
```

Test invalid:

```
Num1: 10
Num2: abc
Traceback (most recent call last):
  File "...", line 3, in <module>
TypeError: unsupported operand type(s) for /: 'str' and 'str'
```

Zero:

```
Num1: 10
Num2: 0
Traceback ... ZeroDivisionError
```

Improved:

```
while True:
    try:
        a = float(input("Enter first number: "))
        b = float(input("Enter second number: "))
        if b == 0:
            raise ValueError("Division by zero not allowed")
        print(f"Result: {a / b}")
        break
    except ValueError as e:
        print(f"Error: {e}. Please enter valid numbers.")
    except KeyboardInterrupt:
        print("\nExiting.")
        break
```

Tests:

Valid:

```
Enter first number: 10
Enter second number: 2
Result: 5.0
```

Invalid:

```
Enter first number: abc
Error: could not convert string to float: 'abc'. Please enter valid numbers.
```

Zero:

```
Enter first number: 10
Enter second number: 0
Error: Division by zero not allowed. Please enter valid numbers.
```